Ex. No.: 5
Date:

System Calls Programming

Aim: To experiment system calls using fork(), execlp() and pid() functions.

Algorithm:

1. **Start**
   - Include the required header files (stdio.h and stdlib.h).
2. **Variable Declaration**
   - Declare an integer variable pid to hold the process ID.
3. **Create a Process**
   - Call the fork() function to create a new process. Store the return value in the pid variable:
     - If fork() returns:
       - -1: Forking failed (child process not created).
       - 0: Process is the child process.
       - Positive integer: Process is the parent process.
4. **Print Statement Executed Twice**
   - Print the statement:

     scss
     Copy code
     THIS LINE EXECUTED TWICE

     (This line is executed by both parent and child processes after fork()).

5. **Check for Process Creation Failure**
   - If pid == -1:
     - Print:

       Copy code
       CHILD PROCESS NOT CREATED

     - Exit the program using exit(0).
6. **Child Process Execution**
   - If pid == 0 (child process):
     - Print:
       - Process ID of the child process using getpid().
       - Parent process ID of the child process using getppid().
7. **Parent Process Execution**
   - If pid > 0 (parent process):
     - Print:
       - Process ID of the parent process using getpid().
       - Parent's parent process ID using getppid().
8. **Final Print Statement**
   - Print the statement:

     objectivec

Copy code
IT CAN BE EXECUTED TWICE

(This line is executed by both parent and child processes).

9. **End**

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
   pid_t pid;

   pid = fork(); // create child process

   if (pid < 0) {
      perror("fork failed");
      return 1;
   }

   if (pid == 0) {
      // Child process
      printf("Child process (PID: %d, Parent PID: %d)\n", getpid(), getppid());
      printf("Child is executing 'ls' command:\n");
      execlp("ls", "ls", "-l", NULL); // replace child with 'ls -l'
      perror("execlp failed"); // only prints if execlp fails
   } else {
      // Parent process
      printf("Parent process (PID: %d, Child PID: %d)\n", getpid(), pid);
      wait(NULL); // wait for child to finish
      printf("Child finished execution.\n");
   }
   return 0;
```

Output:

```
Parent process (PID: 12245, Child PID: 12246)
Child process (PID: 12246, Parent PID: 12245)
Child is executing 'ls' command:
total 20
-rwxr-xr-x 1 user user 16664 May  1 10:22 fork_exec
-rw-r--r-- 1 user user  1023 May  1 10:20 fork_exec.c
Child finished execution.
```

Result:

Program executed successfully using fork(), excelp() and pid().